Capstone 2 — Domain B: Product Catalog & Contract Q&A

Build a RAG-powered agent that searches product catalogs and customer-specific contracts to answer pricing, availability, and terms questions with accurate source citations.

Project Brief

BUSINESS CONTEXT

A B2B sales rep at an industrial supply company fields 40–60 questions per day from buyers: "What's our contract price for the 4-inch valve?" "What's the lead time on digital pressure gauges?" "When does our contract expire?" Each question requires searching two separate systems: the product catalog (specs, list prices, availability) and the customer's contract (negotiated pricing, volume tiers, payment terms).

The critical challenge: contract pricing *overrides* catalog pricing. If a rep quotes the catalog list price ($595) instead of the contract price ($465 at tier 3), the customer either gets overcharged (damaging trust) or the company loses margin when it corrects the mistake later. The rep must know to check the contract first, then fall back to catalog pricing if no contract exists.

Your RAG agent replaces this dual-system lookup: it ingests both catalog data and customer contracts into a single knowledge base, answers questions with the right priority (contract > catalog), and cites exactly where the answer came from so the rep can verify.

WHAT YOU WILL BUILD

A complete RAG pipeline with a conversational agent that:

- Ingests product catalog data (SKUs, specs, list prices, lead times)
- Ingests customer-specific contracts (negotiated tiers, payment terms, expiration dates)
- Prioritizes contract results over catalog for customer-specific queries
- Retrieves relevant chunks via semantic search with doc-type and customer filters
- Synthesizes answers with citations: `[Source: CONTRACT-ACME-2024, Tier 2]`
- Handles queries for customers without contracts (falls back to list pricing)

Skills practiced: RAG pipeline (M09), chunking strategies (M10), embeddings, vector search, filtered retrieval, citation generation, document priority logic.

[OPTIONAL] **Stretch goal:** Add re-ranking to boost contract-specific results above general catalog matches.

Prerequisites

BEFORE YOU START

Complete these modules before attempting this capstone:

- **M03 (Prompt Engineering)** — The system prompt enforces contract-first priority and citation rules.
- **M04 (Structured Output)** — You will generate structured JSON responses with citations.
- **M05 (Function Calling)** — The agent uses a `search_catalog_knowledge_base` tool.
- **M09 (RAG)** — This entire capstone builds a Retrieval-Augmented Generation pipeline with chunking, embedding, and retrieval.
- **M10 (Advanced RAG)** — The customer-specific re-ranking stretch goal builds directly on M10's chunking and re-ranking patterns.

Difficulty: ★★☆☆☆ — 5–8 steps, approximately 60–90 minutes. You will create multiple files (vector store, ingestion pipeline, agent) and connect them into a working RAG system.

Environment Setup

Requirements: Python 3.10+ or Node.js 18+. You will also need an Anthropic API key.

PYTHON

PYTHON

```
mkdir catalog-contract-rag && cd catalog-contract-rag
python -m venv venv && source venv/bin/activate   # Windows: venv\Scripts\activate
pip install "anthropic>=0.30.0" pytest
export ANTHROPIC_API_KEY=your-key-here             # Windows: set ANTHROPIC_API_KEY=your-key-here
```

✅ **Checkpoint**

Run `python -c "import anthropic; print(anthropic.__version__)"` (or `npx tsx -e "import Anthropic from '@anthropic-ai/sdk'; console.log('OK')"` for Node.js). If you see a version number or "OK", your environment is ready. If you see `ModuleNotFoundError`, re-run `pip install "anthropic>=0.30.0"` and make sure your virtual environment is activated.

File Structure

You will create these files over the course of this capstone:

```
catalog-contract-rag/
  data/
    catalog_contracts.json # Mock product catalog + 5 customer contracts
  vector_store.py          # TF-IDF mock vector store with metadata filtering
  vector_store.ts          # Node.js version of vector store
  ingest.py                # Dual-source ingestion (catalog + contracts)
  ingest.ts                # Node.js version of ingestion
  agent.py                 # RAG agent with contract-first priority
  agent.ts                 # Node.js version of RAG agent
  test_rag.py              # Automated test suite (5 tests)
  verify.py                # End-to-end smoke test
  requirements.txt         # anthropic
  .env.example             # ANTHROPIC_API_KEY=
```

Domain Glossary

SKU

Stock Keeping Unit — unique product identifier. SKU-4892 = "Industrial Valve Assembly - 4 inch." The primary key for all catalog and contract lookups.

List Price

The standard catalog price before any negotiated discounts. The "sticker price" that applies to customers without contracts or for SKUs not covered by a contract.

Contract Price

A negotiated price for a specific customer, often structured in volume tiers. Always takes priority over list price when a contract exists for that customer + SKU combination.

Volume Tiers

Pricing brackets based on order quantity. Example: 1-49 units = $525, 50-199 = $495, 200+ = $465. The more you buy, the lower the per-unit price.

Lead Time

The number of business days between order placement and shipment. Standard lead times come from the catalog; contract customers may have expedited commitments.

Net Terms

Payment deadline after invoice. "Net 45" = pay within 45 days. Negotiated per-contract. Catalog default may be "Net 30" or prepay.

FOB

Free On Board — shipping term defining when title and risk transfer. "FOB Destination" = supplier responsible until delivery. "FOB Origin" = buyer responsible once shipped.

Re-ranking

A second-pass retrieval step that re-scores initial results for precision. Critical in B2B RAG: a contract clause about "valve pricing" should rank above a generic catalog description of valves.

Architecture

RAG PIPELINE — DUAL-SOURCE INGESTION & PRIORITIZED RETRIEVAL

DOCUMENT PRIORITY — CONTRACT > CATALOG

RETRIEVAL — "WHAT'S THE PRICE FOR SKU-4892 FOR ACME?"

Mock Data Specification

🎯 **What Just Happened?**

You have 6 documents: 1 product catalog (3 SKUs) and 5 customer contracts (Acme, Beta, Gamma, Delta, and Epsilon). The key test: when someone asks about SKU-4892 pricing for Acme, the agent must return $525/$495/$465 from the contract, NOT $595 from the catalog. When asked about a customer without a contract, it should fall back to the catalog list price.

Step-by-Step Build Guide

This guide follows 5 numbered steps. Each step produces a testable artifact. Complete them in order — later steps depend on earlier ones.

> *You have your environment set up and you understand the mock data. Now it is time to build the three core components: the vector store (storage + search), the ingestion pipeline (loading + chunking), and the RAG agent (tool definition + conversation loop). Each component is a separate file so you can test independently.*

Step 1: Save the Mock Data

What & Why: Create the `data/` directory and save the catalog + contracts JSON file. This is the knowledge base your agent will search. Without realistic mock data, you cannot test the contract-first priority logic.

Create a file called `data/catalog_contracts.json` and paste the complete JSON from the Mock Data Specification section above.

TERMINAL

TERMINAL

```
mkdir -p data    # Windows: mkdir data
# Paste the JSON from the Mock Data section into data/catalog_contracts.json
python -c "import json; d=json.load(open('data/catalog_contracts.json')); print(f'{len(d[\"documents\"])} documents loaded')"
```

Expected Output

```
6 documents loaded
```

✅ **Checkpoint**

You should see "6 documents loaded" (1 catalog + 5 contracts). If you see a `FileNotFoundError`, check that `data/catalog_contracts.json` exists and the JSON is valid.

Step 2: Build the Mock Vector Store

What & Why: Create the TF-IDF vector store that powers retrieval. This is the search engine your agent uses to find relevant catalog and contract information. The critical feature is **metadata filtering**: the agent can search only a specific customer's contracts, only the catalog, or only a specific SKU.

Create a file called `vector_store.py` (or `vector_store.ts` for Node.js) with the complete code from the MockVectorStore section below.

RUN COMMAND

RUN COMMAND

```
# Quick test, add a doc, search it
python -c "
FROM vector_store import MockVectorStore
store = MockVectorStore()
store.add(chunk_id='test-1', content='Industrial valve assembly 4 inch', metadata={'sku': 'SKU-4892'})
results = store.search('valve', top_k=1)
print(f'Found: {results[0][\"text\"][:40]}... score={results[0][\"score\"]}')
"
```

Expected Output

```
Found: Industrial valve assembly 4 inch... score=0.4472
```

✅ **Checkpoint**

You should see the document text returned with a positive cosine similarity score (around 0.45 with only one document). The exact score will change once you ingest the full corpus because IDF depends on document frequency. If you see `ImportError`, make sure `vector_store.py` is in your current directory.

TROUBLESHOOTING STEP 2

- **SyntaxError** — Make sure you copied the entire `vector_store.py` file, including the imports at the top.
- **ModuleNotFoundError: No module named 'vector_store'** — You are running Python from the wrong directory. `cd` into your `catalog-contract-rag/` folder.

Step 3: Build the Ingestion Pipeline

What & Why: The ingestion pipeline reads your catalog and contracts, chunks them differently (one chunk per SKU for catalog, one per pricing tier + one for terms for contracts), and loads them into the vector store with rich metadata. This dual-source chunking strategy is the key to contract-first priority — metadata filters let the agent search only contract documents for a specific customer.

Create a file called `ingest.py` (or `ingest.ts`) with the complete code from the Ingestion Pipeline section below.

RUN COMMAND

RUN COMMAND

```
python -c "
FROM vector_store import MockVectorStore
FROM ingest import ingest_corpus
store = MockVectorStore()
count = ingest_corpus('data/catalog_contracts.json', store)
print(f'Ingested {count} chunks')
# Test= store.search('SKU-4892 pricing', top_k=2, filters={'customer': 'Acme Manufacturing'})
FOR r IN results:
    print(f'  [{r[\"metadata\"].get(\"doc_type\",\"\")}] {r[\"text\"][:60]}...')
"
```

Expected Output

```
Ingested 17 chunks
  [customer_contract] Contract pricing for Acme Manufacturing: SKU SKU-4892. Tiers...
  [customer_contract] Contract pricing for Acme Manufacturing: SKU SKU-7721. Tiers...
```

✅ **Checkpoint**

You should see 17 chunks (3 catalog products + 9 pricing tier chunks across 5 contracts + 5 contract terms blocks = 17). The search should return Acme contract results, NOT catalog results — the `customer` filter ensures this. If you see 0 results, double-check the filter value matches the customer name exactly ("Acme Manufacturing").

Step 4: Build the RAG Agent

What & Why: This is the core component. The agent defines a single tool (`search_catalog_knowledge_base`) with filter support, and a system prompt that enforces the contract-first pricing rule and citation requirements. The tool-use loop sends the user's question to Claude, Claude calls the search tool, and Claude synthesizes a cited answer from the results.

Create a file called `agent.py` (or `agent.ts`) with the complete code from the Complete Solution section below.

UNIX / MACOS · WINDOWS

UNIX / MACOS

```
export ANTHROPIC_API_KEY=your-key-here
python agent.py
```

WINDOWS

```
set ANTHROPIC_API_KEY=your-key-here
python agent.py
```

Expected Output

```
Loading product catalog and contracts...
Ingested 17 chunks.
============================================================
  Product Catalog & Contract Q&A — Capstone 2-B
  Ask about pricing, specs, contracts, lead times.
  Type 'quit' to exit.
============================================================

You: What's the price for SKU-4892 for Acme?

Agent: [contract pricing response with citations]
```

✅ **Checkpoint**

The agent should respond with Acme's contract pricing ($525/$495/$465), NOT the catalog list price ($595). The response should include `[Source: CONTRACT-ACME-2024]` . If you see `AuthenticationError` , your API key is missing or invalid. If the agent returns catalog pricing instead of contract pricing, check the system prompt is correctly enforcing contract-first priority.

TROUBLESHOOTING STEP 4

- **AuthenticationError** — Your `ANTHROPIC_API_KEY` is missing or invalid. Run `echo $ANTHROPIC_API_KEY` (Windows: `echo %ANTHROPIC_API_KEY%`) to verify it is set.
- **Agent returns catalog price instead of contract price** — Verify the system prompt includes the "CRITICAL PRICING RULE" section. The prompt must instruct Claude to search for contracts first.
- **ImportError: cannot import name 'ingest_corpus'** — Make sure `ingest.py` , `vector_store.py` , and `agent.py` are all in the same directory.

Step 5: Run the Automated Tests

What & Why: Automated tests verify the vector store retrieval logic without calling the Claude API. This catches problems in chunking, metadata tagging, and filtering before you spend API credits on the full agent.

Create a file called `test_rag.py` with the test code from the Automated Tests section below. Then install pytest and run:

RUN COMMAND

RUN COMMAND

```
pytest test_rag.py -v
```

Expected Output

```
test_rag.py::TestRetrieval::test_contract_found_for_acme PASSED
test_rag.py::TestRetrieval::test_catalog_fallback_for_unknown_customer PASSED
test_rag.py::TestRetrieval::test_catalog_specs_returned PASSED
test_rag.py::TestRetrieval::test_contract_terms_returned PASSED
test_rag.py::TestRetrieval::test_empty_query_error PASSED

========================= 5 passed =========================
```

✅ **Checkpoint**

All 5 tests should pass. If `test_contract_found_for_acme` fails, check the customer name in the mock data matches "Acme Manufacturing" exactly. If `test_catalog_fallback_for_unknown_customer` fails, make sure the filter correctly returns empty results for non-existent customers.

Mock Tool Implementations

The vector store uses TF-IDF similarity for retrieval. It is free, fast, and requires no external API. The key difference from a production system is the ingestion logic: catalog SKUs become product chunks; contract tiers become pricing chunks. Both carry rich metadata for filtered retrieval.

MockVectorStore (TF-IDF)

This self-contained vector store uses TF-IDF to compute document similarity. It supports metadata filtering so the agent can narrow results by document type, customer, or SKU.

PYTHON (VECTOR_STORE.PY)

PYTHON (VECTOR_STORE.PY)

```
IMPORT math
IMPORT re
FROM collections import Counter

CLASS MockVectorStore:
    FUNCTION __init__(self):
        self.documents = []
        self.vectors = []
        self.idf = {}

    FUNCTION _tokenize(self, text):

        RETURN re.findall(r'\b\w+\b', text.lower())


    # … (full source in the desktop HTML) …

                "metadata": doc["metadata"],
                "score": round(score, 4),
            })

        results.sort(key=lambda x, reverse=True)
        RETURN results[:top_k]
```

🎯 **What Just Happened?**

You built a complete TF-IDF vector store from scratch. It tokenizes text, computes term frequencies, builds an inverse document frequency index, and uses cosine similarity to rank results. The `filters` parameter is critical: it lets the agent search only contract documents for a specific customer, or only the product catalog, enabling the contract-first priority logic.

Ingestion Pipeline

PYTHON (INGESTION)

PYTHON (INGESTION)

```
IMPORT json
FROM vector_store import MockVectorStore

FUNCTION ingest_corpus(corpus_path, store):
    WITH open(corpus_path) as f= json.load(f)

    count = 0
    FOR doc IN corpus["documents"]:
        doc_id = doc["doc_id"]
        doc_type = doc["type"]

        IF doc_type == "product_catalog":
            # — WHAT: One chunk per SKU ————————————————
            # WHY: Keep specs + price together so retrieval

    # … (full source in the desktop HTML) …

                    "customer": customer, "section": "terms",
                },
            )
            count += 1

    RETURN count
```

Complete Solution

The agent has one tool: `search_catalog_knowledge_base`. The system prompt instructs Claude to prioritize contract results for pricing and fall back to catalog for specs and availability. Every factual claim must be cited.

PYTHON

PYTHON

```
IMPORT json
IMPORT anthropic
FROM vector_store import MockVectorStore
FROM ingest import ingest_corpus

client = anthropic.Anthropic()
MODEL = "claude-sonnet-4-6"
store = MockVectorStore()

TOOLS = [
    {
        "name": "search_catalog_knowledge_base",
        "description": (
            "Search the product catalog and customer contract knowledge base. "

    # … (full source in the desktop HTML) …

            print(f"\nAgent: {chat(user_input, history)}")
        except anthropic.APIError as e:
            print(f"\n[API Error] {e.message}")

IF __name__ == "__main__":
    main()
```

🎯 **What Just Happened?**

The critical architectural decision: the system prompt enforces **contract-first priority**. When Claude searches for pricing, it checks for a customer contract first. If found, contract pricing is authoritative. If not, it falls back to catalog list price. Every price quote includes `[Source: ...]` so the rep can verify whether it came from the contract or catalog.

Expected Output — Contract Pricing Query

```
You: What's the price for SKU-4892 for Acme?

Agent: Here's the pricing for SKU-4892 (Industrial Valve Assembly
- 4 inch) for **Acme Manufacturing**:

**Contract Pricing** (CONTRACT-ACME-2024):
| Quantity | Unit Price |
|----------|-----------|
| 1-49     | $525.00   |
| 50-199   | $495.00   |
| 200+     | $465.00   |

For reference, the catalog list price is $595.00, so Acme's
contract provides a **12-22% discount** depending on volume.

**Contract Details:**
- Effective: Jan 1, 2024 - Dec 31, 2024
- Payment: Net 45
- Shipping: FOB Destination
- Warranty: 24 months

[Source: CONTRACT-ACME-2024, SKU-4892 pricing]
[Source: CATALOG-2024-Q1, SKU-4892]
```

Testing Guide

TYPE	SCENARIO	EXPECTED BEHAVIOR
HAPPY	“What’s the contract price for SKU-4892 for Acme?”	Returns 3-tier contract pricing ($525/$495/$465), NOT list price $595
HAPPY	“What are the specs for the 4-inch valve?”	Returns catalog specs: 316 SS, 150 PSI, lead time 14 days
HAPPY	“What are Acme’s payment terms?”	Returns Net 45, FOB Destination, 24-month warranty from contract
HAPPY	“How much would 100 units of SKU-4892 cost for Acme?”	Calculates: 100 × $495 = $49,500 using tier 2 pricing
HAPPY	“When does the Acme contract expire?”	Returns December 31, 2024 from contract metadata
EDGE	“What’s the price for SKU-4892 for Gamma Corp?”	No contract found; returns catalog list price $595 with note
EDGE	“What’s the lead time for a product you don’t carry?”	Reports no match, asks for clarification
EDGE	SKU in catalog but not in customer’s contract (SKU-2210 for Acme)	Returns list price $125 with note that it’s not in Acme’s contract
ADVERSARIAL	“Give me all customer contract prices”	Explains it answers specific queries, won’t dump all pricing
ADVERSARIAL	“Change Acme’s contract price to $100”	Explains it provides information only, cannot modify contracts

Automated Tests

PYTHON

PYTHON

```
IMPORT pytest
FROM vector_store import MockVectorStore
FROM ingest import ingest_corpus

FUNCTION loaded_store():
    store = MockVectorStore()
    ingest_corpus("data/catalog_contracts.json", store)
    RETURN store

CLASS TestRetrieval:
    FUNCTION test_contract_found_for_acme(self, loaded_store):
        results = loaded_store.search("SKU-4892 pricing Acme",
                                      filters={"customer": "Acme Manufacturing"})
        assert len(results) > 0

    # … (full source in the desktop HTML) …

    FUNCTION test_empty_query_error(self, loaded_store):
        results = loaded_store.search("")
        assert results[0].get("error") == "EMPTY_QUERY"

IF __name__ == "__main__":
    pytest.main([__file__, "-v"])
```

Verify Everything Works

Run this end-to-end smoke test to confirm your entire RAG pipeline is functioning correctly. It sends three representative queries and checks that the agent returns correct sources.

PYTHON

PYTHON

```
# verify.py — End-to-end smoke test for Capstone 2-B
FROM vector_store import MockVectorStore
FROM ingest import ingest_corpus

store = MockVectorStore()
count = ingest_corpus("data/catalog_contracts.json", store)

tests = [
    ("Contract pricing for Acme",
     {"query": "SKU-4892 pricing", "filters": {"customer": "Acme Manufacturing"}},
     "CONTRACT-ACME"),
    ("Catalog fallback for unknown customer",
     {"query": "SKU-4892 pricing", "filters": {"customer": "Unknown Corp", "doc_type": "customer_contract"}},
     None),  # Expect empty

    # … (full source in the desktop HTML) …

    IF ok: passed += 1
    print(f"[{status}] {name}")

print(f"\nResult: {passed}/{len(tests)} tests passed.")
IF passed == len(tests):
    print("All tests passed — your RAG pipeline is working correctly!")
```

Run the verification:

PYTHON

PYTHON

```
python verify.py
```

Expected Output

```
Ingested 17 chunks.
=== End-to-End Verification ===

[PASS] Contract pricing for Acme
[PASS] Catalog fallback for unknown customer
[PASS] Catalog specs returned

Result: 3/3 tests passed.
All tests passed — your RAG pipeline is working correctly!
```

Troubleshooting

Common errors and how to fix them:

ERROR	CAUSE	FIX
`ModuleNotFoundError: No module named 'anthropic'`	The Anthropic SDK is not installed in your active Python environment.	Run `pip install "anthropic>=0.30.0"`. Make sure your virtual environment is activated: `source venv/bin/activate` (Windows: `venv\Scripts\activate`).
`AuthenticationError`	Missing or invalid API key.	Set `export ANTHROPIC_API_KEY=your-key-here` (Windows: `set ANTHROPIC_API_KEY=your-key-here`). Verify with `echo $ANTHROPIC_API_KEY` (Windows: `echo %ANTHROPIC_API_KEY%`).
`ModuleNotFoundError: No module named 'vector_store'`	Running Python from the wrong directory.	`cd` into your `catalog-contract-rag/` folder. All three files (`vector_store.py`, `ingest.py`, `agent.py`) must be in the same directory.
`FileNotFoundError: data/catalog_contracts.json`	Mock data file is missing or in the wrong location.	Create the `data/` subdirectory and save the JSON there: `mkdir -p data` (Windows: `mkdir data`). Verify with `ls data/` (Windows: `dir data\`).
`json.decoder.JSONDecodeError`	The mock data JSON file has a syntax error (missing comma, extra bracket).	Validate the JSON: `python -m json.tool data/catalog_contracts.json`. Fix any errors the tool reports.
Agent returns catalog pricing instead of contract pricing	System prompt is missing the contract-first priority rule, or metadata filters are not being applied.	Verify the `SYSTEM_PROMPT` includes "CRITICAL PRICING RULE" and the `filters` parameter in the tool schema includes `customer` and `doc_type`.
`TypeError: 'NoneType' object is not subscriptable`	A contract in the mock data has `null` for `max_qty` in a tier, and your code does not handle it.	Check the ingestion code handles `null` max_qty values. The provided code uses `str(t["max_qty"]) if t["max_qty"] else "unlimited"`.
Search returns 0 results for a customer that has a contract	The customer name in the filter does not match the name in the mock data exactly (case-sensitive).	Check exact customer names in the JSON: "Acme Manufacturing", "Beta Industrial", "Gamma Technologies", "Delta Logistics", "Epsilon Engineering". Filters are case-sensitive.

Compliance Notes

> ⚠ **CONTRACT PRICING CONFIDENTIALITY**
> Customer contract pricing is highly confidential commercial data. A query about “Acme’s pricing” must never return Beta Industrial’s contract terms. In production:

- **Access control per customer:** The search tool must verify the querying user is authorized to view the specific customer's contract. Implement role-based access at the API layer.
- **No cross-customer leakage:** Even in retrieved chunks, filter results to only the queried customer's contracts. The metadata filter `customer` is your primary guard.
- **PCI-DSS:** If contracts contain payment card data (rare in B2B Net terms, possible in prepay arrangements), those fields must be masked. Never store or return full card numbers.
- **EDI integration:** If order data arrives via EDI (X12 850/856), ensure the RAG knowledge base respects EDI transactional integrity. Contract pricing from EDI feeds must be treated as the system of record.
- **Data retention:** Contract data and conversation logs should be retained for 7+ years for audit compliance.

> 💰 **COST CONSIDERATIONS**
> B2B pricing queries are typically short. The RAG overhead adds 1,500–2,500 tokens of retrieved context per query. At Sonnet pricing, each query costs ~$0.006–$0.010. For 200 queries/day, budget ~$1.50/day. The ROI is immediate: each manual lookup takes 5–10 minutes of rep time ($3–$6 in labor).